Python

Story behind the name Python



- The programming language **Python** is **not named after the snake**. It was named after the British comedy television series **Monty Python's Flying Circus**.
- Python was created by Guido van Rossum in the late 1980s at the Dutch research institute CWI. He began developing Python around **December 1989**.
- Guido wanted a name that was **"short, unique, and slightly mysterious.“**
- When Python was first released in **1991**, many people assumed it was named after the snake. Guido has repeatedly clarified that the inspiration was **Monty Python**, not reptiles. The snake has simply become the language's unofficial mascot over time.
- Try out: import this

Python Code Execution

A simple demonstration of variables, conditions, and operations in Python

student_records.py

```python
student_count = 0
print(student_count)

# 60 students enrolled
student_count += 60

if student_count > 0:
print("Class should commence.")
else:
print("Course to be withdrawn.")

students = "Student " * student_count
print(students)
```

Let's unpack the code

Understanding variable assignment and dynamic typing in Python

student_records.py

```
student_count = 0 # Focus
print(student_count)

# 60 students enrolled
student_count += 60

if student_count > 0:
print("Class should commence.")
else:
print("Course to be withdrawn.")
```

Line 1 Details

Variable Assignment

We create a variable named `student_count` and assign it the value `0` using the assignment operator `=`.

01

No Explicit Declaration

Python variables are created the moment you first assign a value to them. You do not need to declare them beforehand.

02

Dynamic Typing

No need to specify data types. Python automatically infers the type, and you can freely reassign variables to different types later.

Let's unpack the code

Understanding function calls and how the print function operates in Python

student_records.py

```
student_count = 0
print(student_count) # Focus
# 60 students enrolled
student_count += 60
if student_count > 0:
print("Class should commence.")
else:
print("Course to be withdrawn.")
```

Line 2 Details

Function Calls

The `print()` function is a built-in Python tool that displays information on the screen. We execute it by writing parentheses directly after its name.

01

Passing Arguments

Inputs passed to a function inside the parentheses are called ***arguments***. Here, we pass `student_count` as the argument to print its current value.

02

Screen Output

Once called, the function takes the evaluated value of the argument and outputs it directly to the system console or standard output.

Let's unpack the code

Understanding comments and variable reassignment in Python

student_records.py

```
student_count = 0
print(student_count)
# 60 students enrolled # Focus
student_count += 60
if student_count > 0:
print("Class should commence.")
else:
print("Course to be withdrawn.")
```

Lines 3 & 4 Details

Comments & Reassignment

We document our code using comments and update existing variables using mathematical assignment operators.

01

Code Comments

In Python, comments begin with the # symbol. The interpreter completely ignores these lines, making them perfect for documentation.

02

In-Place Reassignment

Reassigning the value of an existing variable looks just like creating a new one. It uses the = assignment operator, often combined with arithmetic like +=.

Let's unpack the code

Understanding conditional statements and code blocks in Python

student_records.py

```
student_count = 0
print(student_count)
# 60 students enrolled
student_count += 60
if student_count > 0:
print("Class should commence.")
else:
print("Course to be withdrawn.")
students = "Student " * student_count
print(students)
```

Lines 7 - 10 Details

Conditional Statements

Python uses conditional code execution to branch program logic based on dynamic evaluations.

01 Code Block & Indentation

The colon (`:`) indicates a new code block starts. Subsequent lines must be indented to be part of that block.

02 The Else Branch

The `else` block runs as a fallback only when the preceding `if` conditions evaluate to False.

Let's unpack the code

Understanding operator overloading and string multiplication in Python

student_records.py

```
student_count = 0
print(student_count)
# 60 students enrolled
student_count += 60
if student_count > 0:
print("Class should commence.")
else:
print("Course to be withdrawn.")
students = "Student " * student_count # Focus
print(students)
```

Lines 9 & 10 Details

Operator Overloading

Python allows operators to have different behaviors depending on the type of data they are applied to.

01

String Multiplication

Using the * operator with a string and an integer repeats the string. Here, `"Student "` is repeated 60 times to create a single long string.

02

Polymorphic Behavior

Operators like * and + are overloaded, meaning their function changes dynamically based on whether operands are numbers, strings, or lists.

Try it out

Execute these expressions in your Python environment to observe their outputs

sandbox.py

```python
# 1. Inspecting variables
type(student_count)

# 2. Division operations
print(15 / 2)
print(15 // 2)

# 3. Aggregations
print(max(15, 30, 27))
print(min(15, -30, 27))
```

Core Concepts

Key Observations

Predict what each line of code will produce before running it in your terminal.

01

Type Inspection

The `type()` function returns the class category of an object (e.g., integer, float, string).

02

Division vs. Floor Division

Notice how `/` results in a decimal, while `//` truncates the fractional part.

03

Extremes Evaluation

The `max()` and `min()` helper functions easily process comma-separated values.